

AI in images and video processing

Thomas de Jaeger
thomas.de-jaeger@epita.fr

Class details

- 21h: 3h*7
- Grade: 40% Labs
60% Final Project
- All the material will be online: https://github.com/tdejaeger/Epita_lectures
- For anonymous feedback: [Feedback - Google Docs](#)

Class organisation

- Lectures
- Labs during class
- Project

Final Project Overview — Tumor Detection in Medical Images

- **Project Goal:** Build a complete AI pipeline that detects the presence of tumors in medical images (e.g., brain MRIs) using image processing and deep learning techniques.

- **What You Will Do:**

1. Data Cleaning & Preprocessing

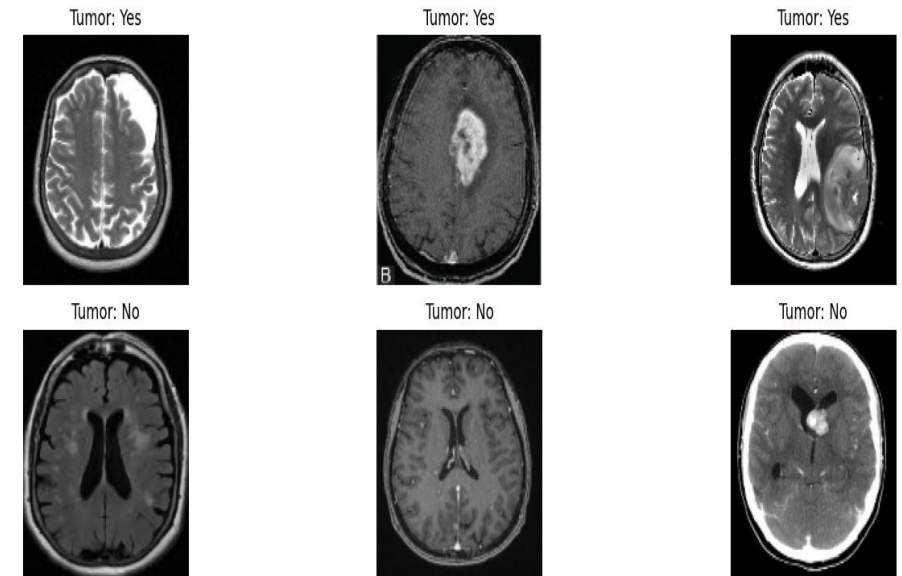
- Remove corrupt images
- Standardize orientation
- Resize and normalize images

2. Exploratory Analysis

- Visualize examples
- Understand class distribution

3. Image Processing with OpenCV

- Grayscale conversion, filtering (blur, threshold)
- Optional: region isolation (tumor contour detection)



Final Project Overview — Tumor Detection in Medical Images

- **What You Will Do:**

- 4. Model Training with PyTorch

- Train a CNN classifier (tumor vs no tumor)
- Use transformations: normalization, data augmentation
- Evaluate: accuracy, confusion matrix, predictions
- If tumor, localize it

- 5. Visualization & Reporting

- Show model predictions on sample test images
- Present results, explain pipeline, discuss limitations

Deliverables:

- Jupyter Notebook, google colab
- Oral presentation to explain the code and the choice
- Report pdf

Deadline:

- Thursday 10th July 2025

Syllabus

- **Session 1:** Introduction & Basics of Computer Vision
- **Session 2:** Filtering, Edge Detection & Shape Detection
- **Session 3:** Intro to PyTorch + CNNs for Classification
- **Session 4:** Feature Extraction & Segmentation
- **Session 5:** Object Detection & Optical Character Recognition
- **Session 6:** Advanced Topics: Facial & Emotion Recognition, Tracking
- **Session 7:** Project presentation

Environment & Requirements

Tools & Libraries to Install

Library	Purpose	Session(s)
numpy	Array and matrix operations	All
matplotlib	Plotting and displaying images and histograms	All
opencv-python	Image I/O and processing (main library used)	1–5, 7
scikit-learn	For clustering (k-means, SVM)	4, 6, 7
torch	Deep learning (CNN, classifiers, etc.)	3, 4, 6,
torchvision	Datasets and transforms for vision tasks	3, 4, 6,
mediapipe	Hand/face detection in real-time	4
pytesseract	OCR for text recognition	5

You can run this course on **Google Colab** or **locally**.

Google Colab

What is Google Colab? Google Colab is a free cloud service that supports Jupyter notebooks and provides free access to computing resources including GPUs.

To use it:

- Log in with a Google account
- Open a .ipynb notebook from Drive or GitHub, or create a new one
- To enable GPU: Runtime > Change runtime type > GPU

Why we use it:

- Free and powerful
- No local install needed
- Familiar interface for Jupyter users

Upload files as a drive from your local machine

```
from google.colab import files  
input_file = files.upload()
```


Introduction & Basics of Computer Vision

Session 1

Introduction & Basics of Computer Vision

Session 1

Objectives:

- Understand how images are represented digitally (pixels, color channels)
- Learn about common color spaces: RGB, BGR, HSV, Grayscale
- Understand image histograms and histogram equalization
- Practice basic image manipulations with OpenCV

What is Computer Vision?

Definition:

Computer vision is a field of AI that enables computers to interpret and process visual data such as images and videos.

Why using IA ?

- Human vision is intuitive, but limited by fatigue, perception, and speed.
- Computer Vision can:
 - Analyze thousands of images/videos rapidly
 - Detect patterns invisible to humans
 - Eliminate subjectivity in analysis

What is Computer Vision?

Human Vision vs Computer Vision

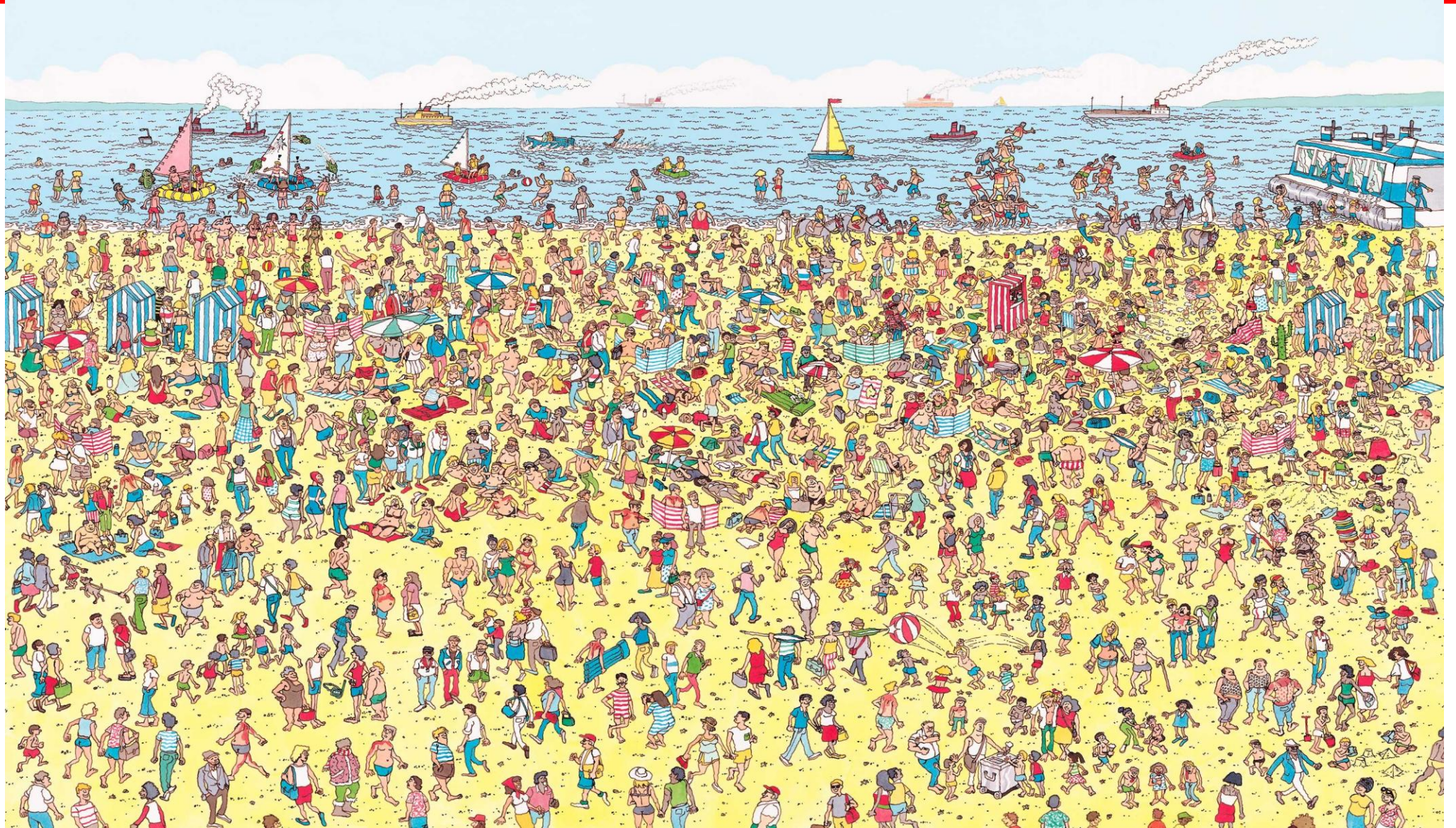


Where is Computer Vision used?

Applications across fields:

- Medical (tumor detection, retinal analysis)
- Automotive (autonomous driving)
- Security (face recognition)
- Industry (defect detection)
- Agriculture, Finance, Telecom, etc

Where is Waldo?



Where is Waldo?



AutoML Vision **BETA** waldos + ADD IMAGES LABEL STATS EXPORT DATA

IMAGES TRAIN EVALUATE PREDICT

All images 26
Labeled 26
Unlabeled 0

Type to filter...
waldo 19
waldobody 7
Add label

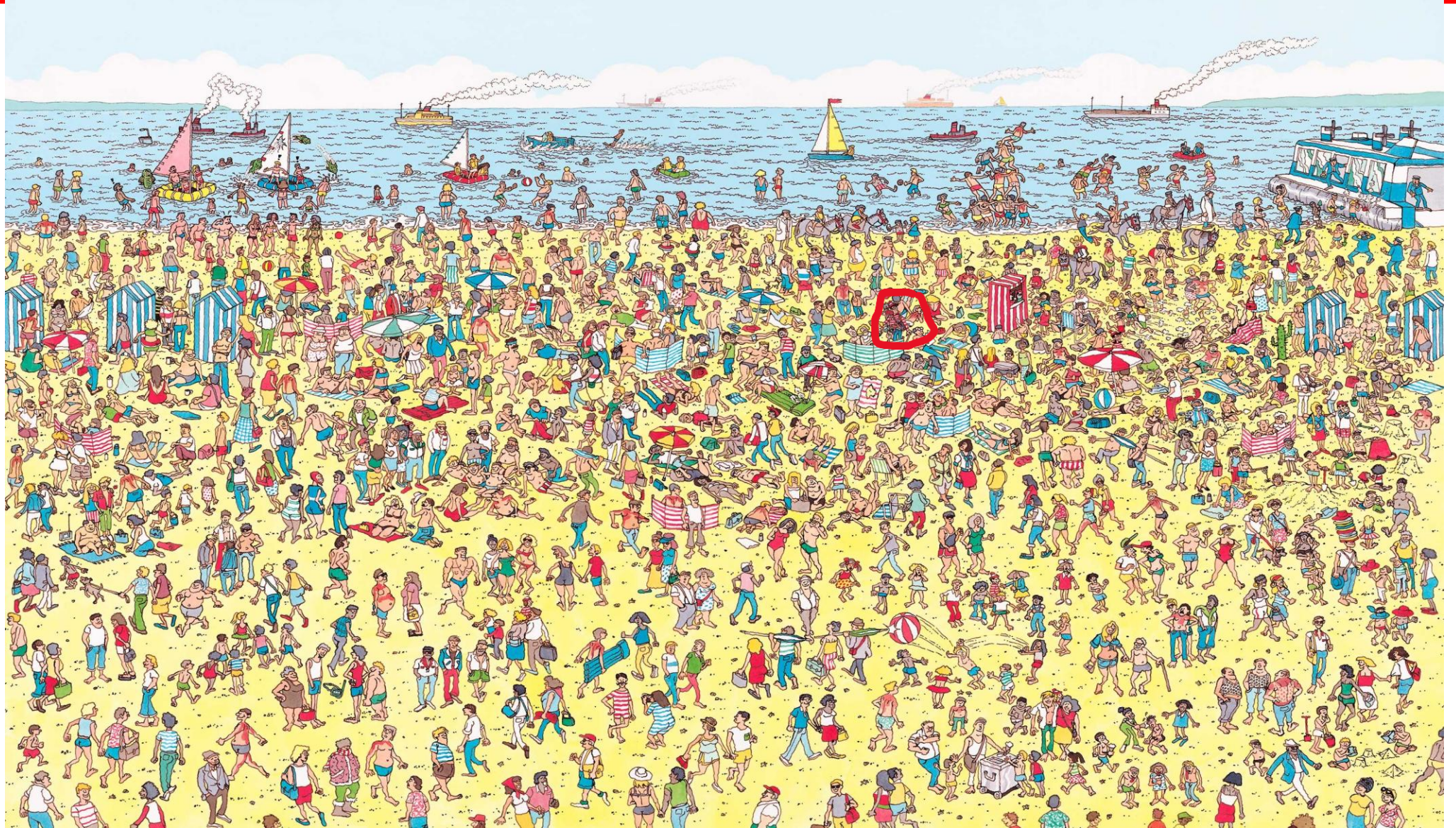
2 selected Label Delete

waldo waldo waldo waldo waldo

waldo waldo waldo waldo waldo

waldo waldo waldo waldo waldo

Where is Waldo?



How Computer vision works?

Three Key Steps:

Computer Vision System



Input



Sensing Device

Acquire Data:

Capture visual input (camera, video, sensors)



Intrepreing Device

Train Models:

Use labeled data to learn visual patterns

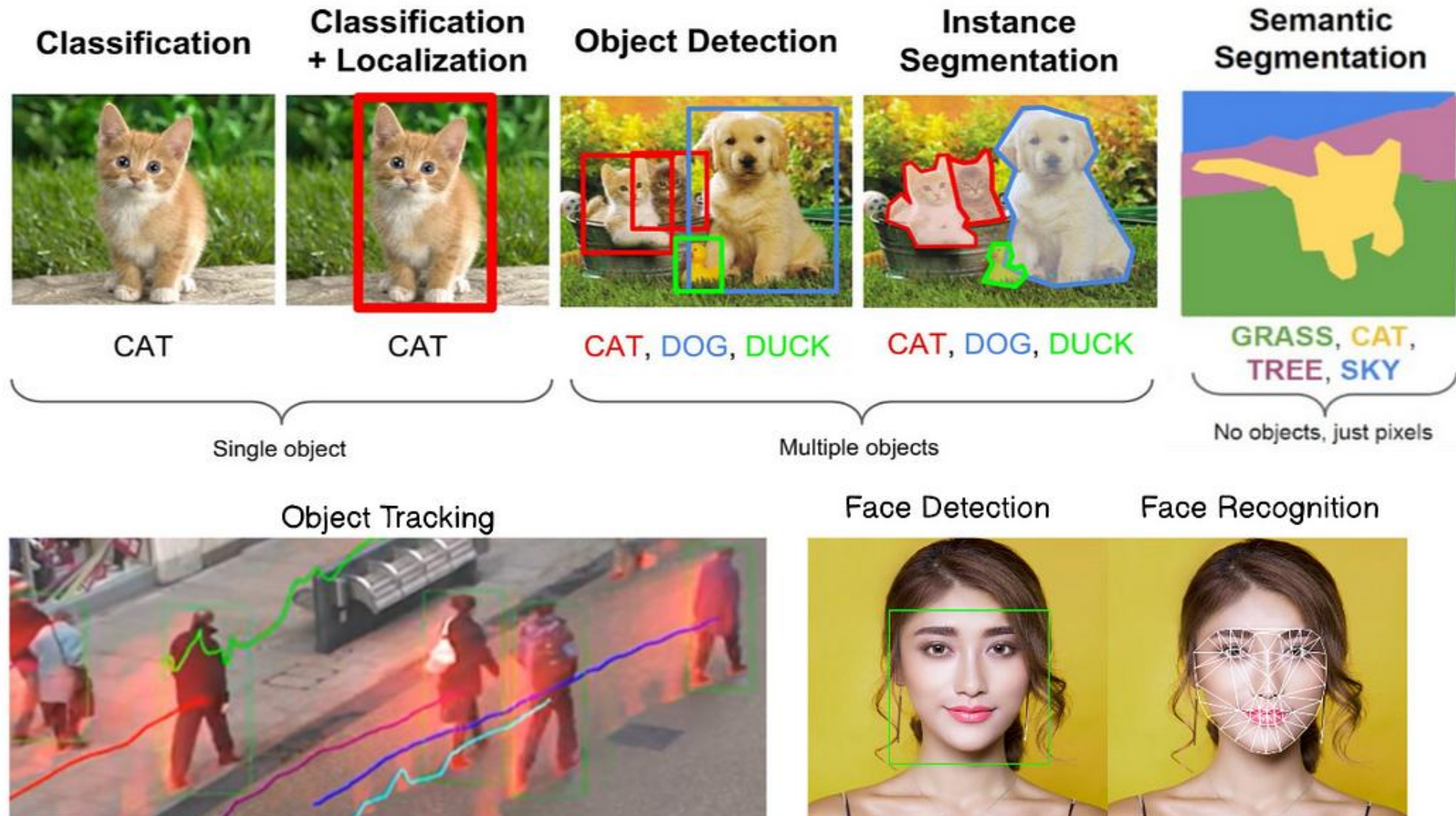
**Bananas,
Grapes &
Strawberry**

Output

Make Predictions:

Apply trained model to detect/analyze objects.

What can you do with CV?



What is a image?

**What eye see for a gray scale
image:**



What is a image?

If we zoom:

→ you will see some small square boxes on this image

→ **Pixels**

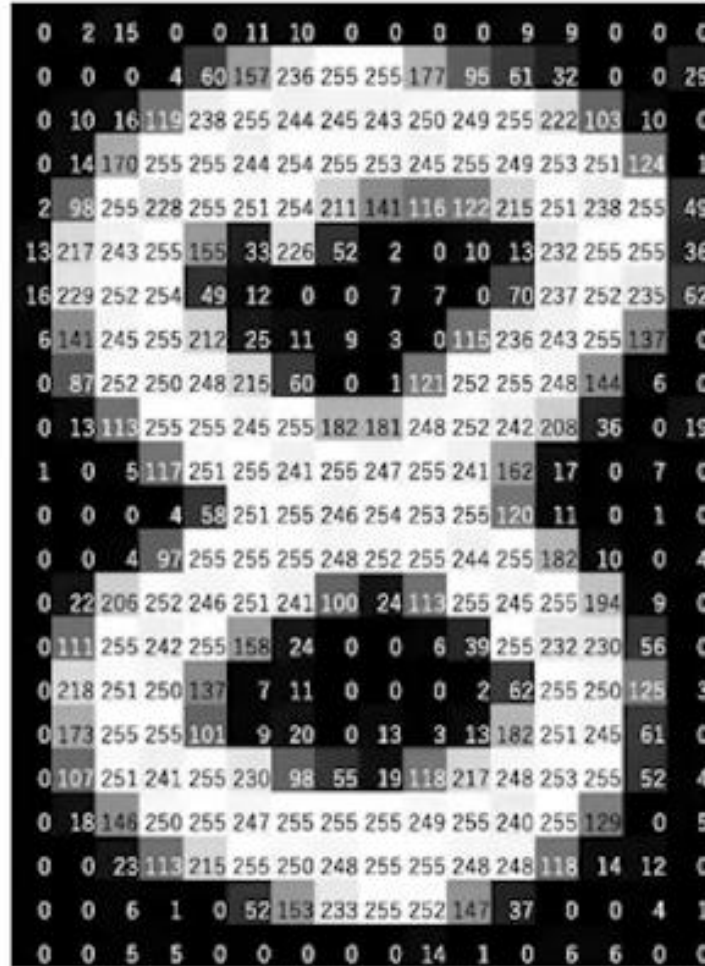
It is the unit of measurement for the size and quality of a digital image



What is a image?

What a computer sees:

- Images are represented as a 2D or 3D matrix of pixels
- In grayscale: Each pixel has intensity [0–255] (0 = black, 255 = white)



What happens to color images?



Colour Image

What happens to color images?

Almost **all colors** can be generated from the three primary colors – **Red, Green, and Blue**



Colour Image

=



Red

+



Green

+



Blue

What happens to color images?

Almost **all colors** can be generated from the three primary colors – **Red, Green, and Blue**

141	142	143	144	145
151	152	153	154	155
161	162	163	164	165
171	172	173	174	175
181	182	183	184	185
191	192	193	194	195

R

					141	142	143	144	145
					151	152	153	154	155
					161	162	163	164	165
35	36	37	38	39	173	174	175		
45	46	47	48	49	183	184	185		
55	56	57	58	59	193	194	195		
65	66	67	68	69					
75	76	77	78	79					
85	86	87	88	89					

R

G

--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

R

G

B

For the computer one image has the dimension of N (width) * M (height) * 3 (RGB)

Algorithms extract features (edges, color regions) from pixel patterns

OpenCV Image Manipulation Essentials

Read/Display Using OpenCV

■ `cv2.imread('filename', flag)` : A function that reads an image and turns it into a Numpy object

▶ flag options:

`cv2.IMREAD_COLOR` (1) : load a color image. Any transparency of image will be neglected. It is the default flag.

`cv2.IMREAD_GRAYSCALE` (0) : load image in grayscale mode

`cv2.IMREAD_UNCHANGED` (-1) : load image including alpha channel

`cv2.IMREAD_ANYDEPTH` : Use precision as 16/32 bits or 8 bits, depending on the image

`cv2.IMREAD_ANYCOLOR` : 3 channels available, used as color images

`cv2.IMREAD_REDUCED_GRAYSCALE_2` : 1 channel, 1/2 size, grayscale application

`cv2.IMREAD_REDUCED_GRAYSCALE_4` : 1 channel, 1/4 size, grayscale application

`cv2.IMREAD_REDUCED_GRAYSCALE_8` : 1 channel, 1/8 size, grayscale application

`cv2.IMREAD_REDUCED_COLOR_2` : 3 channels, 1/2 size, using BGR images

`cv2.IMREAD_REDUCED_COLOR_4` : 3 channels, 1/4 size, using BGR images

`cv2.IMREAD_REDUCED_COLOR_8` : 3 channels, 1/8 size, using BGR images

■ `cv2.imshow('windowname', image)` : A function that shows a specific image to the window name

■ `cv2.imwrite('filename.png/jpg/...', image)` : Save image as a file

■ `cv2.cvtColor(image, flag)` : A function to change the color shape of an image

▶ flag options:

`cv2.COLOR_BGR2RGB` or 4 : change BGR to RGB

`cv2.COLOR_BGR2GRAY` or 6 : change BGR to Gray

`cv2.COLOR_RGB2GRAY` or 7 : change RGB to Gray

`cv2.COLOR_GRAY2RGB` or 8 : change Gray to RGB

OpenCV Image Manipulation Essentials

- **Resize:**

`resized = cv2.resize(img, (200, 300))` # Resize to specific dimensions

`resized = cv2.resize(img, None, fx=0.5, fy=0.5)` # Resize by scale

Explanation:

- (200, 300) means you explicitly set width = 200 and height = 300.
- None means OpenCV ignores fixed size and uses scaling factors:
- `fx=0.5, fy=0.5` resizes the image to 50% of its original width and height

OpenCV Image Manipulation Essentials

- **Resize:**

```
resized = cv2.resize(img, (200, 300))      # Resize to specific dimensions  
resized = cv2.resize(img, None, fx=0.5, fy=0.5)  # Resize by scale
```

- **Crop:**

```
cropped = img[50:200, 100:300] # y1:y2, x1:x2.
```

- **Rotate:**

```
(h, w) = img.shape[:2]          # Get image height and width  
center = (w // 2, h // 2)      # Compute the center  
M = cv2.getRotationMatrix2D(center, 45, 1.0) # 45° rotation, scale=1  
rotated = cv2.warpAffine(img, M, (w, h))  # Apply the rotation
```

Explanation:

- Scale factor (1.0: original size)
- Angle (Positive angle → Counter-clockwise)
- `fx=0.5, fy=0.5` resizes the image to 50% of its original width and height

OpenCV Image Manipulation Essentials

- **Resize:**

`resized = cv2.resize(img, (200, 300))` # Resize to specific dimensions

`resized = cv2.resize(img, None, fx=0.5, fy=0.5)` # Resize by scale

- **Crop:**

`cropped = img[50:200, 100:300]` # y1:y2, x1:x2.

- **Rotate:**

`(h, w) = img.shape[:2]` # Get image height and width

`center = (w // 2, h // 2)` # Compute the center

`M = cv2.getRotationMatrix2D(center, 45, 1.0)` # 45° rotation, scale=1

`rotated = cv2.warpAffine(img, M, (w, h))` # Apply the rotation

- **Flip:**

`flip_h = cv2.flip(img, 1)` # Horizontal

`flip_v = cv2.flip(img, 0)` # Vertical

`flip_both = cv2.flip(img, -1)` # Vertical & horizontal

- **SAVE**

`cv2.imwrite("output.jpg", img)`

OpenCV Drawing Functions

- **Rectangle**

`cv2.rectangle(img, (x1, y1), (x2, y2), (255, 0, 0), 2)`

- **Circle**

`cv2.circle(img, (x, y), radius, (0, 255, 0), thickness)`

(255, 0, 0): color as a tuple
> 0 Line thickness in pixels
-1 Fill the shape entirely

- **Arrowed Line**

`cv2.arrowedLine(img, (x1, y1), (x2, y2), (0, 0, 255), 2)`

- **Line**

`cv2.line(img, (x1, y1), (x2, y2), (255, 255, 0), 3)`

- **Ellipse**

`cv2.ellipse(img, (x, y), (axis1, axis2), angle, 0, 360, (255, 0, 255), 2)`

- **Text**

`cv2.putText(img, "Hello CV", (10, 500), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 255), 2): Font size`

Color Masking & Bitwise Operations

Understand how to isolate specific regions of interest using binary masks and bitwise operations in OpenCV.

Key Commands:

- **cv2.inRange(src, lowerb, upperb):** Creates a binary mask from a range of values (e.g. in HSV)
- **cv2.bitwise_and(src1, src2, mask=None):** Combines two arrays or applies a mask (e.g. to keep only certain regions)
- **cv2.bitwise_or(src1, src2, mask=None):** Combines two arrays (union of their non-zero pixels)
- **cv2.bitwise_not(src):** Inverts an array (turns 0s into 255s and vice versa)
- **cv2.bitwise_xor(src1, src2, mask=None):** Keeps only the non-overlapping (exclusive) regions of two masks or arrays: Keeps only non-overlapping areas between two masks/images

HSV Color Space

The HSV color model is an alternative to the RGB color model, designed to be more aligned with how humans perceive and categorize colors.

Hue (H): Color type and is the angle on the color wheel. 0° represents red

Saturation (S): Intensity or purity of the color. 0% = desaturated (gray), 100% = fully saturated (pure color with no gray mixed in). As saturation increases, the color becomes more vibrant.

Value (V): This represents the brightness of the color. It ranges from 0% (black) to 100% (full brightness)

Note that in OpenCV, the HSV color model different

- Hue (H): **Range: 0 to 179**
- Saturation (S): **Range: 0 to 255**
- Value (V): Range: 0 (black) to 255 (full brightness)

hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

Why?

- Separates color (hue) from intensity (value)
- Easier to isolate colors for detection or masking

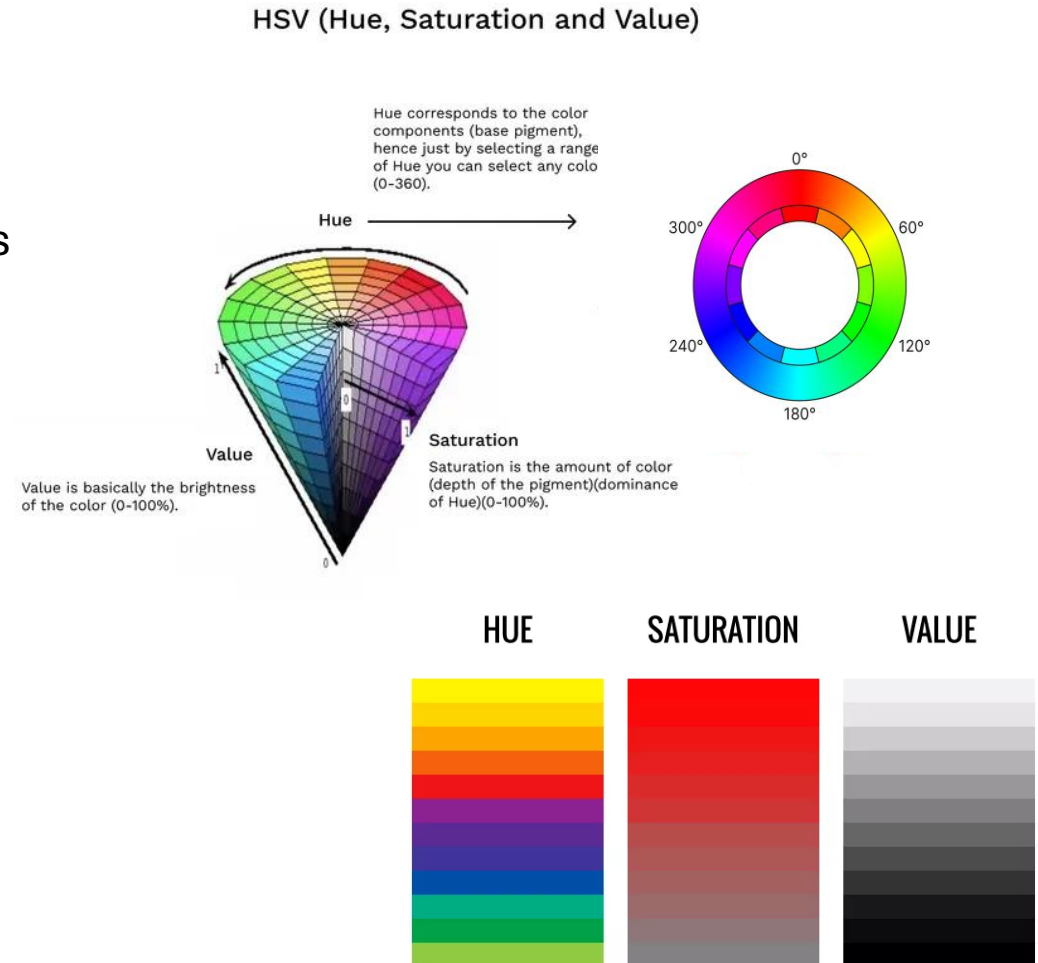


Image histogram

An image histogram is a graphical representation of the distribution of pixel intensities in an image.

The horizontal axis shows intensity values:

- Left = dark areas (shadows)
- Center = mid-tones
- Right = bright areas (highlights)

The vertical axis shows the number of pixels for each intensity level.

Use cases:

Detect if an image is underexposed (too dark) or overexposed (too bright)

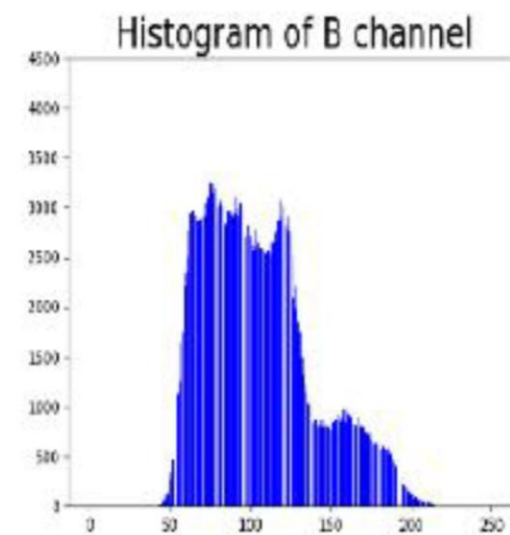
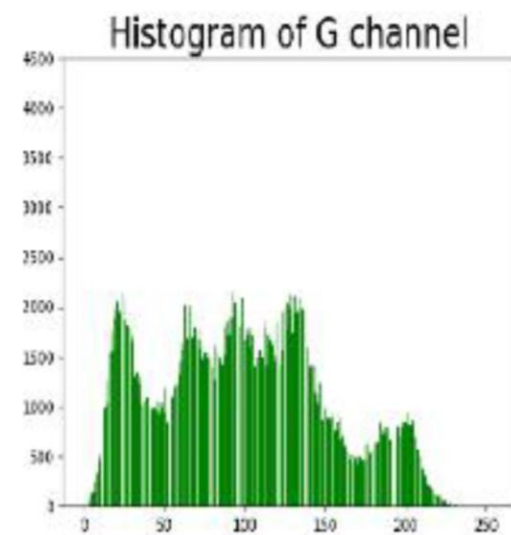
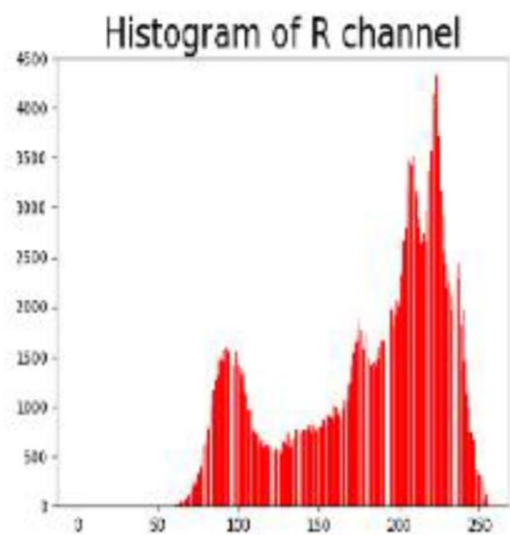
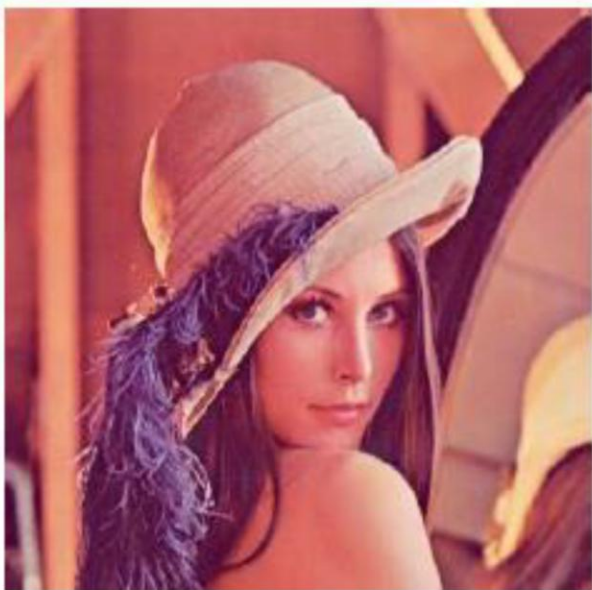
Evaluate image contrast

Improve visibility using histogram equalization

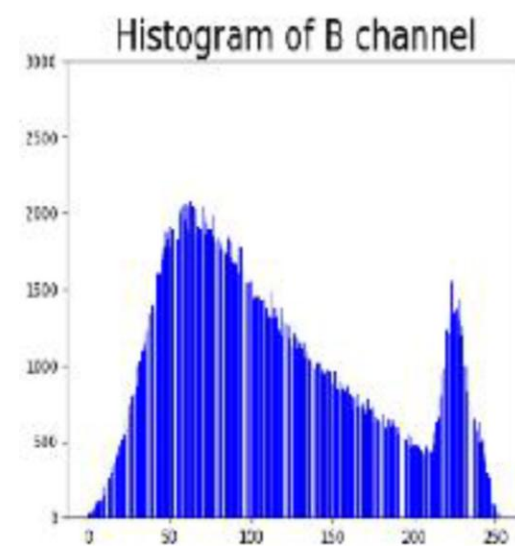
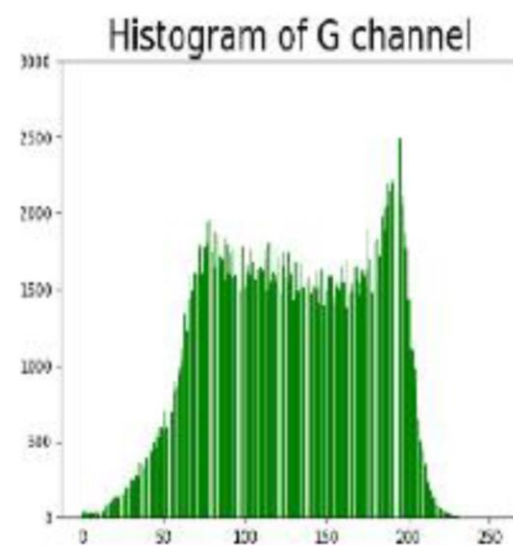
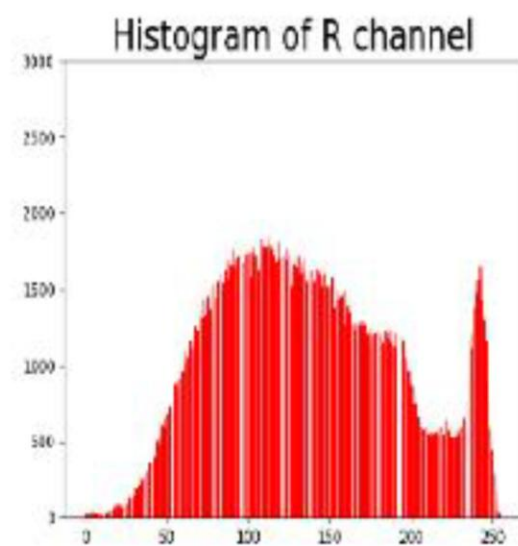
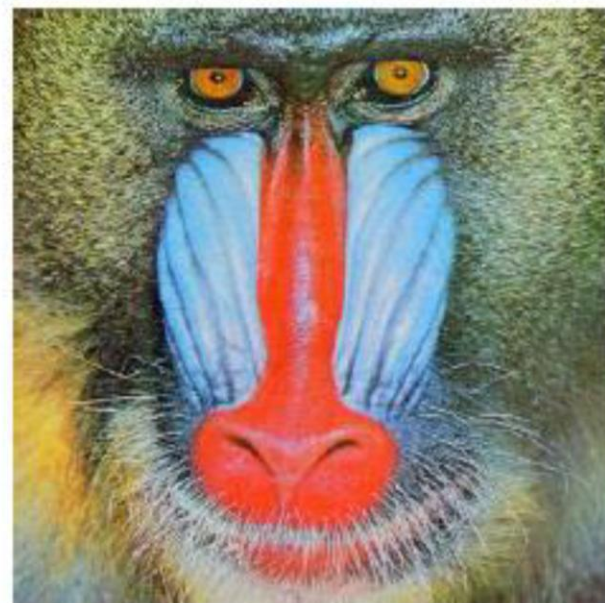
Helps with: exposure detection, thresholding, contrast enhancement

Grayscale: 0 (black) to 255 (white)

Color: histograms per channel (R, G, B)



(a)



(b)

Histogram equalization

Histogram equalization is a contrast enhancement technique that spreads out the most frequent intensity values in an image, making it easier to see features in both bright and dark areas.

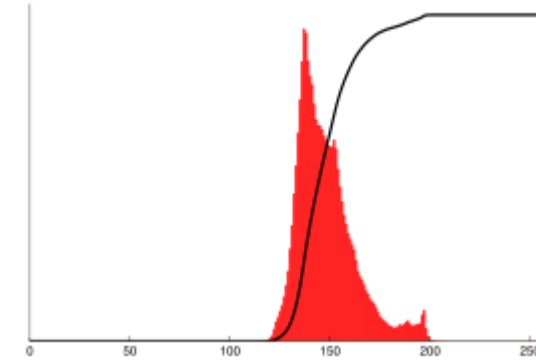
Why use it?

- Improve the contrast in low-contrast images
- Make dark areas lighter and bright areas more defined
- Useful for preprocessing before edge detection or object recognition

How it works:

- Computes the histogram of the image
- Calculates the cumulative distribution function (CDF)
- Redistributes pixel values to use the full range of intensities

Histogram equalization



Before Histogram Equalization

Histogram (red) and cumulative histogram (black)

After Histogram Equalization



Histogram (red) and cumulative histogram (black)



Histogram equalization

Standard Histogram Equalization

- Work best on uniformly grayscale images.
- Affects the entire image globally.
- Can over-amplify noise in certain cases.

equalized = cv2.equalizeHist(img)

CLAHE (Contrast Limited Adaptive Histogram Equalization):

- Applies histogram equalization locally on small regions (tiles)
- Prevents over-amplification of noise by clipping the histogram
- Better results on images with varying lighting or fine details (e.g., medical images)
- CLAHE usually provides smoother and more natural contrast.

clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))

cl1 = clahe.apply(gray)

clipLimit : Limits the contrast amplification.

High value = more contrast, more noise

Low value = smoother, safer enhancement

tileGridSize : Defines the size of the grid for dividing the image into tiles ((4, 4) — finer granularity)

For color images?

You should not apply histogram equalization (standard or CLAHE) directly on RGB channels, because:

- It may distort colors and create unnatural results.
- RGB channels are interdependent, so modifying them independently breaks their color balance.

Instead, first separate the brightness of the image from the color and then run HE:

- Convert the image to LAB color space
- L: Lightness
- A, B: Color components
- Apply CLAHE to the L channel only

Common Image Formats in Computer Vision

Format	Description	Key Properties
JPG	Compressed, very common	Lossy compression, small size, no alpha channel
PNG	Web-friendly, transparency support	Lossless compression, supports alpha channel
TIFF	Scientific/medical use	Lossless, high bit depth, supports metadata
BMP	Raw, uncompressed format	Very large files, rarely used today
DICOM	Medical imaging format	Includes metadata (patient, modality, etc.)

Definitions:

- **Lossy Compression:** Reduces file size by discarding some image data (e.g., JPEG). Results in minor quality loss.
- **Lossless Compression:** Compresses without losing any data. Can fully reconstruct the original image (e.g., PNG, TIFF).
- **Alpha Channel:** Extra channel in an image that represents transparency (e.g., PNG with 4 channels: R, G, B, Alpha).

LAB

IA_images_Lab1_questions